JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface WatchService

All Superinterfaces:

`AutoCloseable`, `Closeable`

```
public interface WatchService
extends Closeable
```

A watch service that *watches* registered objects for changes and events. For example a file manager may use a watch service to monitor a directory for changes so that it can update its display of the list of files when files are created or deleted.

A `Watchable` object is registered with a watch service by invoking its `register` method, returning a `WatchKey` to represent the registration. When an event for an object is detected the key is *signalled*, and if not currently signalled, it is queued to the watch service so that it can be retrieved by consumers that invoke the `poll` or `take` methods to retrieve keys and process events. Once the events have been processed the consumer invokes the key's `reset` method to reset the key which allows the key to be signalled and re-queued with further events.

Registration with a watch service is cancelled by invoking the key's `cancel` method. A key that is queued at the time that it is cancelled remains in the queue until it is retrieved. Depending on the object, a key may be cancelled automatically. For example, suppose a directory is watched and the watch service detects that it has been deleted or its file system is no longer accessible. When a key is cancelled in this manner it is signalled and queued, if not currently signalled. To ensure that the consumer is notified the return value from the `reset` method indicates if the key is valid.

A watch service is safe for use by multiple concurrent consumers. To ensure that only one consumer processes the events for a particular object at any time then care should be taken to ensure that the key's `reset` method is only invoked after its events have been processed. The `close` method may be invoked at any time to close the service causing any threads waiting to retrieve keys, to throw `ClosedWatchServiceException`.

File systems may report events faster than they can be retrieved or processed and an implementation may impose an unspecified limit on the number of events that it may accumulate. Where an implementation *knowingly* discards events then it arranges for the key's `pollEvents` method to return an element with an event type of `OVERFLOW`. This event can be used by the consumer as a trigger to re-examine the state of the object.

When an event is reported to indicate that a file in a watched directory has been modified then there is no guarantee that the program (or programs) that have modified the file have completed. Care should be taken to coordinate access with other programs that may be updating the file. The `FileChannel` class defines methods to lock regions of a file against access by other programs.

Platform dependencies

The implementation that observes events from the file system is intended to map directly on to the native file event notification facility where available, or to use a primitive mechanism, such as polling, when a native facility is not available. Consequently, many of the details on how events are detected, their timeliness, and whether their ordering is preserved are highly implementation specific. For example, when a file in a watched directory is modified then it may result in a single `ENTRY_MODIFY` event in some implementations but several events in other implementations. Short-lived files (meaning files that are deleted very quickly after they are created) may not be detected by primitive implementations that periodically poll the file system to detect changes.

If a watched file is not located on a local storage device then it is implementation specific if changes to the file can be detected. In particular, it is not required that changes to files carried out on remote systems be detected.

Implementation Note:

The JDK's `WatchService` implementations buffer up to 512 pending events for each registered watchable object. If this limit is exceeded, pending events are discarded and the special `OVERFLOW` event is queued. This special event is the trigger to re-examine the state of the object, e.g. scan a watched directory to get an updated list of the files in the directory. The limit for the pending events can be changed from its default with the system property `jdk.nio.file.WatchService.maxEventsPerPoll` set to a value that parses as a positive integer. This may be useful in environments where there is a high volume of changes and where the impact of discarded events is high.

Since:

1.7

See Also:

`FileSystem.newWatchService()`

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description

void	**close**()	Closes this watch service.
WatchKey	**poll**()	Retrieves and removes the next watch key, or null if none are present.
WatchKey	**poll**(long timeout, TimeUnit unit)	Retrieves and removes the next watch key, waiting if necessary up to the specified wait time if none are yet present.
WatchKey	**take**()	Retrieves and removes next watch key, waiting if none are yet present.

Method Details

close

```
void close()
    throws IOException
```

Closes this watch service.

If a thread is currently blocked in the take or poll methods waiting for a key to be queued then it immediately receives a ClosedWatchServiceException. Any valid keys associated with this watch service are invalidated.

After a watch service is closed, any further attempt to invoke operations upon it will throw ClosedWatchServiceException. If this watch service is already closed then invoking this method has no effect.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Closeable

Throws:

IOException - if an I/O error occurs

poll

```
WatchKey poll()
```

Retrieves and removes the next watch key, or `null` if none are present.

Returns:

the next watch key, or `null`

Throws:

`ClosedWatchServiceException` - if this watch service is closed

poll

```
WatchKey poll(long timeout,
              TimeUnit unit)
       throws InterruptedException
```

Retrieves and removes the next watch key, waiting if necessary up to the specified wait time if none are yet present.

Parameters:

`timeout` - how long to wait before giving up, in units of unit

`unit` - a `TimeUnit` determining how to interpret the timeout parameter

Returns:

the next watch key, or `null`

Throws:

`ClosedWatchServiceException` - if this watch service is closed, or it is closed while waiting for the next key

`InterruptedException` - if interrupted while waiting

take

```
WatchKey take()
       throws InterruptedException
```

Retrieves and removes next watch key, waiting if none are yet present.

Returns:

the next watch key

Throws:

`ClosedWatchServiceException` - if this watch service is closed, or it is closed while waiting for the next key

`InterruptedException` - if interrupted while waiting

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.